Network on Chip Generation with LLMs

Arteris IP - INRIA
ugo.lecerf@arteris.com

January 9, 2026

1 Introduction

Modern chips are composed of the combination of many on-chip components, referred to as IPs (intellectual property blocks). These different IP blocks which are compute cores, memories, flow control units, etc. are integrated together to form a System-on-Chip (SoC). In order to transit data between the different IP blocks with satisfactory performance and cost, an on-chip interconnect, or Network-on-Chip (NoC) is required. The NoC is responsible for transiting data, power, control, flow, debug and clock signals (amongst others) across the entire chip. For this reason, NoC (Network-on-Chip) designs require careful planning, and are vital in determining the final cost-performance performance of an ASIC chip.

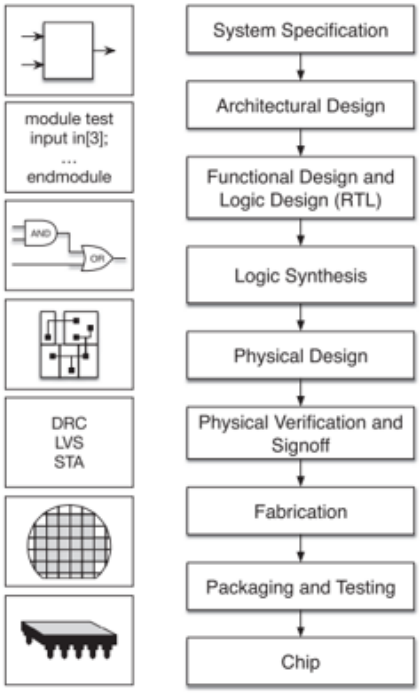


Figure 1: Design flow for an ASIC chip

Figure 1 shows the different steps required for end-to-end chip design.

Arteris IP sells on-chip components to SoC manufacturers. The main component sold by Arteris is a NoC, presented to clients in the form of RTL (register-transfer-level model). Given a system specification, we must first come up with a functional design architecture which satisfies the requirements for the IP component on the chip. For the interconnect, the architecture design phase defines the nodes

and edges of a directed graph which transits data packets across the on-chip network to each of the on-chip components.

In the context where chips are becoming increasingly complex, designing an efficient interconnect is a challenging task. Much like establishing a transport network in a city, we must balance both cost and capacity. For this reason hardware manufacturers have always used EDA (Electronic Design Automation) tools to help design complex systems. Generally, the design of a routing network is related to the *minimum spanning Steiner tree problem*, which is NP-hard. Given the rise of AI tools, and their capacity to handle high-dimensional environments, we may hope that they can help guide hardware designers in generating the on-chip interconnects.

2 Problem Statement

Figure 2 shows input and output representations of the architecture design phase. Initiators and Target units are connected together through segments, and packets are routing between the different segments through *switches* which are responsible for packet routing. Using switches allows us to re-use the same wires over multiple initiator-target routes, reducing the total wirelength of the design.

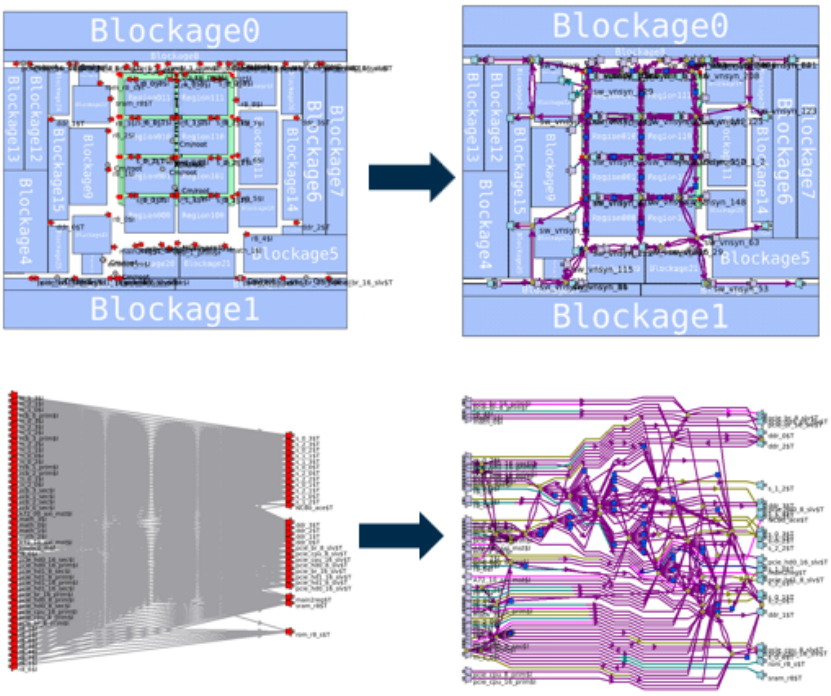


Figure 2: Example network architecture synthesis, from floorplan and connectivity requirements. There are the blockages and connectivity requirements (top-left, bottom-left, respectfully) as inputs to our problem, and we wish to design a network connecting all elements (represented physically in the top-right figure, logically in the bottom-right).

There are a few important architecture constraints that come from the system specifications:

1. **Blockages** are areas on the chip's floorplan representing the area taken up by existing components. This defines which areas of the floorplan are free space, and can be used to place elements.

2. The **connectivity** tells us which initiator components would like to send data packets to which target components. This is a *directed bipartite graph*, where initiators send packets to targets.

Collectively, we refer to initiators and targets as Network Interface Units (NIUs). A single initiator-target pair defines a **route** that must exist, and be valid, in the network.

3. The resulting graph must be **deadlock-free**. Meaning that it must be impossible for packets to get stuck in an un-resolvable "traffic jam".

The above are *hard constraints* that must be respected in order for the architecture design to be valid. Otherwise, designers seek to optimize and balance out certain costs:

1. **Wirelength** refers to the total length of wires in the design. This is related to the cost of printing the design, as well as wire congestion on the chip so we generally aim to reduce the wirelength of the design.

2. **Route-length** refers to the total length travelled by routes in the network. This is related to the average latency in the network.

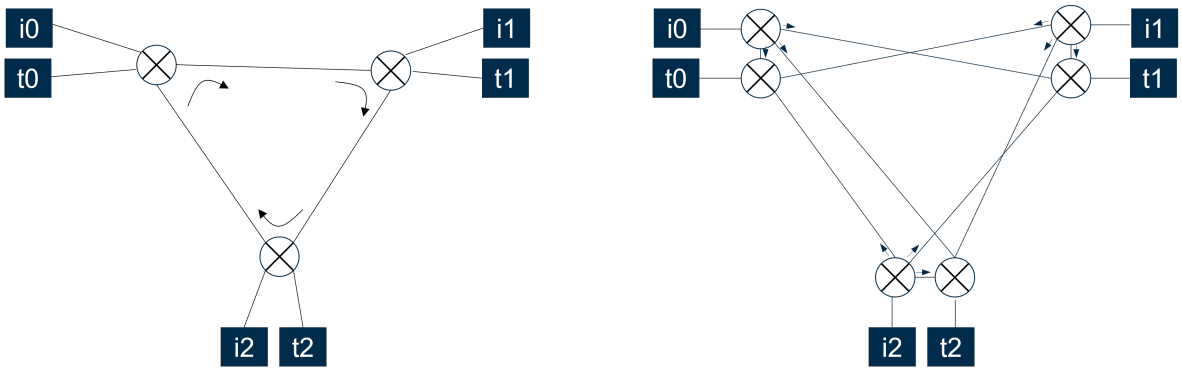
These two criteria are in opposition to each other; a minimum spanning tree will optimize wirelength at the cost of making the average packet travel further on average, whereas a fully connected graph will optimize the average latency of packets at the cost of having a lot of wires.

2.1 Avoiding deadlocks

We can look at an example with 3 initiators, and 3 targets. Let's say each of the initiators would like to send packets to each of the targets. Hence we must ensure the validity of 9 routes:

$$(i_0 \to t_0), (i_0 \to t_1), (i_0 \to t_2), (i_1 \to t_0), ..., (i_2 \to t_2)$$

Figure 3a shows how a possible solution. However in that case, there is a cycle in the routing path, meaning that the design is prone to deadlocks. The design in figure 3b prevents this by adding switches, removing any cycles in the routing graph at the cost of increased wirelength.



(a) deadlock-prone network

(b) deadlock-free network

Figure 3: Example networks

2.2 Generating Networks with Language Models

There already exists classical solvers for generating such networks, most being related to the minimum spanning Steiner tree problem (a variant of the minimum spanning tree problem, where we are allowed to add additional nodes to the tree). For large designs, these solvers can become quite slow, and so we seek to use statistical approaches and heuristics which can re-use past experiences to better design networks, just as human designers naturally do.

For this reason, our goal is to investigate how well LLMs can take design specifications, and output a valid architecture which resembles past network architectures. A challenging feature of this approach is that solutions are based on a *combination* of both *physical* (can only place network elements in the floorplan's free space) and *logical* (must connect all routes, avoiding deadlocks) constraints. Independently there exist models which are well-tuned to handle either of these constraint modalities, however combining them often proves difficult.

2.3 Goal

Using a dataset of network architectures that have already been generated by a classic solver, our goal is to fine-tune an LLM base model to reproduce the network architecture, from the specifications. The specifications include initiators and target positions, connectivity, floorplan dimension, and blockages. The output network architecture includes the switches that have been added, along with the routing paths that each route takes to get from the initiator to the target unit.

The output of the fine-tuned model can be evaluated according to a few criteria:

1. **Training loss** of the model, where we aim to reproduce as closely as possible the complete networks from the training data.
2. **Validity** of the output networks. Implicitly, when training the model should learn to produce valid outputs (i.e. all routes are connected, switches are not placed in illegal zones, and the network is deadlock-free). However, in fine-tuning this will not be initially part of the training loss, and so it will be interesting to first quantify the validity of the designs alongside the main training loss, and possibly eventually combine this criteria in the model's training objective.

There will be a secondary goal to explore **data representation**. The provided specifications are complete, however we may expect that certain formats, or additional information in the models' context might result in better training and/or generalization. This may include pre-computing some network statistics, including additional information on the floorplan's physical constraints, or simply finding a better way to code the network architecture for which the LLM attention mechanism might be better-suited for.

For the logical constraints of validity, free-space, and absence of deadlocks, it is likely that modifying the data representation, or having inference being done in multiple steps might be a good candidate for improving your results.

3 Dataset

You will be provided with an initial dataset of around 1k small-ish networks which have been generated by a classical solver. Since these can be synthetically generated, we can both increase the dataset size, or generate larger networks if need be.

Here is an example text description for an interconnect architecture:

```
# example NoC description
-- Arch Specification --
{
    "inits": {
        "i_0": {
            "x": 900,
            "y": 139
        },
        "i_1": {
            "x": 401,
            "y": 18
        },
```

```
        "i_2": {
            "x": 3,
            "y": 49
        },
        ...
    },
    "targets": {
        "t_0": {
            "x": 787,
            "y": 982
        },
        "t_1": {
            "x": 576,
            "y": 802
        },
        "t_2": {
            "x": 62,
            "y": 620
        },
        ...
    },
    "connectivity": {
        "r_0": [
            "i_0",
            "t_4"
        ],
        "r_1": [
            "i_0",
            "t_3"
        ],
        "r_2": [
            "i_1",
            "t_4"
        ],
        ...
    },
    "floorplan_dim": [
        1000,
        1000
    ],
    "blockages": {
        "b_0": {
            "x": 4,
            "y": 3,
            "width": 197,
            "height": 228
        },
        "b_1": {
            "x": 63,
            "y": 19,
            "width": 835,
            "height": 781
        },
        ...
    }
}
-- Synthesized Network: --
{
```

```
    "switches": {
        "s_0": {
            "x": 875,
            "y": 15
        },
        "s_1": {
            "x": 900,
            "y": 139
        },
        "s_2": {
            "x": 763,
            "y": 880
        },
        ...
    },
    "routing_paths": {
        "r_0": [
            "i_0",
            "s_1",
            "s_6",
            "t_4"
        ],
        ...
    }
}
```


4 Initial Guidance

Base Models
Use base models available on Huggingface. Prefer to use the base versions instead of the instruction-tuned models as this most closely resembles a text-prediction task.

Fine-tuning libraries
There are multiple available fine-tuning libraries. Since computational resources are a determining factor, we recommend starting out with HF's PEFT (parameter-efficient fine-tuning) library which allows compute-efficient model fine-tuning.

Data Representation
The provided text files describe the entire architecture specification and solution. However, feel free to try out alternative representations. It will be interesting to find the optimal representations for the data, which allow language models to best represent the constraints. It may be that adding additional information, such as distances, or pre-computed solutions might help models find correct solutions.